

他のプログラミング言語と同様に、Kotlin では、あるコードが真と評価されるかどうかに基づいて判断を行うことができます。このようなコードは **条件式** と呼ばれます。また、Kotlin ではループを作成したり、ループを反復処理したりすることもできます。

Conditional expressions

条件式

Kotlin では、条件式を判定するために **if** と **when** が用意されています。

if と **when** のどちらを使うか迷った場合は、**when** を使うことをお勧めします。その理由は以下の通りです：

- コードの可読性が向上します。
- 別のブランチを追加しやすくなります。
- コードのミスが減ります。

{style="note"}

If

if を使用するには、条件式を丸括弧 **()** で囲み、結果が真の場合に実行する処理を中括弧 **{ }** で囲みます：

```
fun main() {
//sampleStart
    val d: Int
    val check = true

    if (check) {
        d = 1
    } else {
        d = 2
    }

    println(d)
    // 1
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-if"}

Kotlin には、三項演算子 **condition ? then : else** はありません。その代わりに、**if** を式として使用できます。各処理につきコードが1行のみの場合、中括弧 **{ }** は省略可能です：

```
fun main() {
//sampleStart
    val a = 1
    val b = 2
}
```

```
println(if (a > b) a else b) // Returns a value: 2
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-if-expression"}

When

条件式に複数の分岐がある場合は、**when** を使用します。

when を使用するには:

- 評価したい値を括弧 **()** で囲みます。
- 分岐を中括弧 **{}** で囲みます。
- 各分岐では、**->** を使用して、各チェックと、チェックが成功した場合に実行するアクションを区切ります。

when は、文(ステートメント)としても式としても使用できます。****文(ステートメント)****は何も返しません。その代わり、何らかの処理を実行します。

以下は、**when**を文(ステートメント)として使用した例です：

```
fun main() {
//sampleStart
    val obj = "Hello"

    when (obj) {
        // Checks whether obj equals to "1"
        "1" -> println("One")
        // Checks whether obj equals to "Hello"
        "Hello" -> println("Greeting")
        // Default statement
        else -> println("Unknown")
    }
    // Greeting
//sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-when-statement"}

すべての分岐条件は、いずれかが満たされるまで順次チェックされることに注意してください。したがって、条件を満たす最初の分岐のみが実行されます。

{style="note"}

式は、コードの後の部分で利用できる値を返します。

以下は、**when** 式を使用する例です。この **when** 式の結果は直ちに変数に代入され、その変数は後で **println()** 関数とともに使用されます。

```
fun main() {
    //sampleStart
    val obj = "Hello"

    val result = when (obj) {
        // obj が「1」の場合、result を「one」に設定します
        "1" -> "One"
        // obj が「Hello」の場合、result を「Greeting」に設定します
        "Hello" -> "Greeting"
        // 前述の条件のいずれも満たされない場合、結果を「Unknown（不明）」に設定する
        else -> "Unknown"
    }
    println(result)
    // Greeting
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-when-expression"}

これまでに見てきた `when` の例には、いずれも条件を示すサブジェクト `obj` がありました。しかし、`when` は条件を表すサブジェクトなしで使うこともできます。

この例では、主語を**含まない** `when` 式を使用して、一連のブール式を検証しています。:

```
fun main() {
    val trafficLightState = "Red" // This can be "Green", "Yellow", or "Red"

    val trafficAction = when {
        trafficLightState == "Green" -> "Go"
        trafficLightState == "Yellow" -> "Slow down"
        trafficLightState == "Red" -> "Stop"
        else -> "Malfunction"
    }

    println(trafficAction)
    // Stop
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-when-expression-boolean"}

ただし、同じコードでも、条件を表すサブジェクトを `trafficLightState` にすることも可能です:

```
fun main() {
    val trafficLightState = "Red" // This can be "Green", "Yellow", or "Red"

    val trafficAction = when (trafficLightState) {
        "Green" -> "Go"
        "Yellow" -> "Slow down"
    }
}
```

```

        "Red" -> "Stop"
        else -> "Malfunction"
    }

    println(trafficAction)
    // Stop
}

```

```
{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-when-expression-boolean-subject"}
```

when を条件を表すサブジェクトと組み合わせて使用すると、コードの可読性と保守性が向上します。また、**when** 式で条件を表すサブジェクトを使用すると、Kotlin がすべての可能性を網羅しているかどうかを確認しやすくなります。一方、**when** 式で条件を表すサブジェクトを使用しない場合は、**else** 分岐を明示的に記述する必要があります。

Conditional expressions practice

条件式の演習

```
Exercise 1 {initial-collapse-state="collapsed" collapsible="true" id="conditional-expressions-exercise-1"}
```

```
課題 1 {initial-collapse-state="collapsed" collapsible="true" id="conditional-expressions-exercise-1"}
```

2つのサイコロを振って同じ目が出たら勝利となる、シンプルなゲームを作成してください。**if** を使って、サイコロの目が一致した場合は **You win :)** を、それ以外の場合は **You lose :(** を出力してください。

この演習では、パッケージをインポートして、**Random.nextInt()** 関数を使用してランダムな **Int** を生成できるようにします。パッケージのインポートに関する詳細については、以下を参照してください。 [Packages and imports](#).

```
{style="tip"}
```

サイコロの出目を比較するには、**等号演算子 (==)** を使用してください。

```
|---|---|
```

```

import kotlin.random.Random

fun main() {
    val firstResult = Random.nextInt(6)
    val secondResult = Random.nextInt(6)
    // Write your code here
}

```

```
{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-control-flow-conditional-exercise-1"}
```

|---|---|

```
import kotlin.random.Random

fun main() {
    val firstResult = Random.nextInt(6)
    val secondResult = Random.nextInt(6)
    if (firstResult == secondResult)
        println("You win :)")
    else
        println("You lose :(")
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-control-flow-conditional-solution-1"}

Exercise 2 {initial-collapse-state="collapsed" collapsible="true" id="conditional-expressions-exercise-2"}

課題 2 {initial-collapse-state="collapsed" collapsible="true" id="conditional-expressions-exercise-2"}

when 式を使用して、以下のプログラムを修正し、ゲーム機のボタンの名前を入力した際に、それに対応するアクションが表示されるようにしてください。

Button	Action
A	Yes
B	No
X	Menu
Y	Nothing
Other	There is no such button

|---|---|

```
fun main() {
    val button = "A"

    println(
        // Write your code here
    )
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-control-flow-conditional-exercise-2"}

|---|---|

```
fun main() {
    val button = "A"

    println(
        when (button) {
            "A" -> "Yes"
            "B" -> "No"
            "X" -> "Menu"
            "Y" -> "Nothing"
            else -> "There is no such button"
        }
    )
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-control-flow-conditional-solution-2"}

Ranges

範囲

ループについて説明する前に、ループで反復処理を行う対象となる範囲をどのように設定するかを理解しておくと役立ちます。

Kotlinで範囲を作成する最も一般的な方法は、`..` 演算子を使用することです。たとえば、`1..4` は `1, 2, 3, 4` と同じ意味になります。

終値を含まない範囲を宣言するには、`..<` 演算子を使用します。たとえば、`1..<4` は `1, 2, 3` と同等です。

範囲を逆順で指定するには、`downTo` を使用します。たとえば、`4 downTo 1` は `4, 3, 2, 1` と同じ意味になります。

1 以外のステップで増加する範囲を宣言するには、`step` と、希望する増加値を使用します。たとえば、`1..5 step 2` は `1, 3, 5` と同じ意味になります。

`Char` の範囲に対しても、同様の処理を行うことができます：

- `『a』 .. 『d』` は `『a』, 『b』, 『c』, 『d』` と同等です。
- `『z』 downTo 『s』 step 2` は `『z』, 『x』, 『v』, 『t』` と同等です。

Loops

ループ（繰り返し）

プログラミングにおいて最も一般的な2つのループ構造は、`for` と `while` です。`for` は、一連の値を順に処理してアクションを実行するために使用します。`while` は、特定の条件が満たされるまでアクションを継続するために使用します。

For

範囲に関する新しい知識を活用して、1 から 5 までの数字を順に処理し、その都度数字を出力する `for` ループを作成することができます。

イテレータ(iterator)と範囲を、キーワード `in` とともに括弧 `()` で囲みます。実行したい処理を中括弧 `{}` 内に記述します：

```
fun main() {  
    //sampleStart  
    for (number in 1..5) {  
        // numberはイテレータで、 1..5は範囲  
        print(number)  
    }  
    // 12345  
    //sampleEnd  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-for-loop"}

コレクションはループを使って反復処理することもできます：

```
fun main() {  
    //sampleStart  
    val cakes = listOf("carrot", "cheese", "chocolate")  
  
    for (cake in cakes) {  
        println("Yummy, it's a $cake cake!")  
    }  
    // Yummy, it's a carrot cake!  
    // Yummy, it's a cheese cake!  
    // Yummy, it's a chocolate cake!  
    //sampleEnd  
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-for-collection-loop"}

While

`while` は 2 通りの使い方ができます：

- 条件式が真である間、コードブロックを実行する。(while)
- まずコードブロックを実行し、その後、条件式を判定する。(do-while)

最初のユースケース (while) では：

- whileループを継続させるための条件式は、括弧 `()` 内に記述してください。
- 実行したいアクションを中括弧 `{}` で囲んで追加してください。

以下の例では、[インクリメント演算子 ++](#) を使用して、`cakesEaten` 変数の値を 1 増やしています。

{style="tip"}

```
fun main() {
    //sampleStart
    var cakesEaten = 0
    while (cakesEaten < 3) {
        println("Eat a cake")
        cakesEaten++
    }
    // Eat a cake
    // Eat a cake
    // Eat a cake
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-while-loop"}

2番目のユースケース (`do-while`) では：

- `while`ループを継続させるための条件式は、括弧 `()` 内に記述してください。
- 実行したいアクションを、キーワード `do` とともに中括弧 `{}` で囲んで指定します。

```
fun main() {
    //sampleStart
    var cakesEaten = 0
    var cakesBaked = 0
    while (cakesEaten < 3) {
        println("Eat a cake")
        cakesEaten++
    }
    do {
        println("Bake a cake")
        cakesBaked++
    } while (cakesBaked < cakesEaten)
    // Eat a cake
    // Eat a cake
    // Eat a cake
    // Bake a cake
    // Bake a cake
    // Bake a cake
    //sampleEnd
}
```

{kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-while-do-loop"}

条件式やループに関する詳細や例については、[条件とループ](#)を参照してください。

Kotlin の制御フローの基本を理解したところで、次は独自の [関数](#) の書き方を学びましょう。

Loops practice {completion-point="true"}

ループの演習 {completion-point="true"}

課題 1 {initial-collapse-state="collapsed" collapsible="true" id="loops-exercise-1"}

8切れのピザ1枚ができるまで、ピザの切れ数を数えるプログラムがあります。このプログラムを次の2つの方法でリファクタリングしてください：

- Use a `while` loop.
- Use a `do-while` loop.

|---|---|

```
fun main() {
    var pizzaSlices = 0
    // Start refactoring here
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    println("There's only $pizzaSlices slice/s of pizza :(")
    pizzaSlices++
    // End refactoring here
    println("There are $pizzaSlices slices of pizza. Hooray! We have a whole
pizza! :D")
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-control-flow-loops-exercise-1"}

|---|---|

```
fun main() {
    var pizzaSlices = 0
    while ( pizzaSlices < 7 ) {
        pizzaSlices++
        println("There's only $pizzaSlices slice/s of pizza :(")
    }
    pizzaSlices++
    println("There are $pizzaSlices slices of pizza. Hooray! We have a whole
```

```
    pizza! :D")
}
```

```
{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution 1" id="kotlin-tour-control-flow-loops-exercise-1-solution-1"}
```

```
|---|---|
```

```
fun main() {
    var pizzaSlices = 0
    pizzaSlices++
    do {
        println("There's only $pizzaSlices slice/s of pizza :(")
        pizzaSlices++
    } while ( pizzaSlices < 8 )
    println("There are $pizzaSlices slices of pizza. Hooray! We have a whole
pizza! :D")
}
```

```
{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution 2" id="kotlin-tour-control-flow-loops-exercise-1-solution-2"}
```

```
Exercise 2 {initial-collapse-state="collapsed" collapsible="true" id="loops-exercise-2"}
```

```
課題 2 {initial-collapse-state="collapsed" collapsible="true" id="loops-exercise-2"}
```

Fizz Buzzというゲームをシミュレートするプログラムを作成してください。あなたの課題は、1から100までの数字を順に表示し、3で割り切れる数字を「fizz」という単語に、5で割り切れる数字を「buzz」という単語に置き換えることです。3と5の両方に割り切れる数はすべて、「fizzbuzz」という単語に置き換えなければならない。

forループを使って数を数え、**when**式を使って、各ステップで何を表示するかを決定します。剰余演算子 (%) を使用して、ある数を割ったときの余りを返します。 **等号演算子** (==) を使用して、余りがゼロであるかどうかを確認します。

```
|---|---|
```

```
fun main() {
    // Write your code here
}
```

```
{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-control-flow-loops-exercise-2"}
```

```
|---|---|
```

```
fun main() {
    for (number in 1..100) {
        println(
            when {
                number % 15 == 0 -> "fizzbuzz"
                number % 3 == 0 -> "fizz"
                number % 5 == 0 -> "buzz"
                else -> "$number"
            }
        )
    }
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-control-flow-loops-solution-2"}

Exercise 3 {initial-collapse-state="collapsed" collapsible="true" id="loops-exercise-3"}

課題 3 {initial-collapse-state="collapsed" collapsible="true" id="loops-exercise-3"}

単語のリストがあります。forとifを使って、文字lで始まる単語だけを出力してください。

String 型には、.startsWith() 関数を使ってください。

|---|---|

```
fun main() {
    val words = listOf("dinosaur", "limousine", "magazine", "language")
    // Write your code here
}
```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-control-flow-loops-exercise-3"}

|---|---|

```
fun main() {
    val words = listOf("dinosaur", "limousine", "magazine", "language")
    for (w in words) {
        if (w.startsWith("l"))
            println(w)
    }
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-control-flow-loops-solution-3"}

- [Previous step](#)
- [Next step](#)